

WEEK 6: Visual Analysis Using Python (Matplotlib & Seaborn)

Objective

The objective of Week 6 is to **visually analyze financial data** and present insights in an understandable format using Python visualization libraries. It helps stakeholders easily understand trends, risks, and relationships.

Dataset Used

- **Input file:** week5_finance_cleaned.csv
 - This dataset was generated after completing Week 5 data cleaning.
 - It contains cleaned financial stock market data with no null values or duplicates.
-

Columns in Dataset

Column	Description
date	Trading date
open	Opening price
high	Highest price
low	Lowest price
close	Closing price
volume	Number of shares traded
daily_return	Daily percentage return
name	Stock/company name

STEP 1: Import Libraries

```
import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns
```

Explanation:

- **pandas** → Used for data handling.
 - **numpy** → Used for mathematical operations.
 - **matplotlib** → Used for basic plots.
 - **seaborn** → Used for advanced statistical visualization.
-

STEP 2: Load Cleaned Dataset

```
df = pd.read_csv("week5_finance_cleaned.csv")  
df.head()
```

Explanation:

- `read_csv()` loads the cleaned dataset.
 - `head()` displays the first five records to verify data loading.
-

STEP 3: Convert Date Column

```
df['date'] = pd.to_datetime(df['date'])
```

Explanation:

- Converts the date column into datetime format.
 - Required for time-series analysis and plotting trends.
-

STEP 4: Data Overview

```
df.info()  
df.describe()
```

Explanation:

- `info()` shows data types and missing values.
- `describe()` provides statistical summary such as mean, min, max, etc.

Matplotlib Fundamentals

Line Chart – Stock Price Trend

```
monthly = df.resample('M', on='date')['close'].mean()

plt.figure(figsize=(14,6))

plt.plot(monthly)

plt.title("Monthly Average Closing Price Trend")

plt.xlabel("Month")

plt.ylabel("Average Closing Price")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```

Explanation:

- Data is resampled monthly to avoid clutter.
 - Shows long-term price movement.
 - Helps identify upward or downward market trends.
-

Bar Chart – Average Closing Price by Company

```
avg_price = df.groupby('name')['close'].mean()

plt.figure(figsize=(10,5))

avg_price.plot(kind='bar')

plt.title("Average Closing Price by Company")

plt.xlabel("Company")

plt.ylabel("Average Price")

plt.show()
```

Explanation:

- `groupby()` groups data by stock name.
- Compares average price across companies.
- Helps investors compare asset strength.

Scatter Plot – Volume vs Price

```
plt.figure(figsize=(8,5))  
plt.scatter(df['volume'], df['close'])  
plt.title("Trading Volume vs Closing Price")  
plt.xlabel("Volume")  
plt.ylabel("Closing Price")  
plt.show()
```

Explanation:

- Shows relationship between trading activity and price.
 - Helps identify demand-driven movements.
-

Pie Chart – Profit vs Loss Days

```
df['movement'] = df['daily_return'].apply(  
    lambda x: 'Profit' if x > 0 else 'Loss'  
)  
movement_count = df['movement'].value_counts()  
plt.figure(figsize=(6,6))  
plt.pie(  
    movement_count.values,  
    labels=movement_count.index,  
    autopct='%1.1f%%',  
    startangle=90  
)plt.title("Profit vs Loss Days")  
plt.show()
```

Explanation:

- Creates a new column identifying profit or loss.
- Pie chart shows market sentiment.
- Useful for risk evaluation.

Seaborn Statistical Visualization

Histogram – Daily Return Distribution

```
plt.figure(figsize=(8,5))  
sns.histplot(df['daily_return'], kde=True)  
plt.title("Distribution of Daily Returns")  
plt.xlabel("Daily Return")  
plt.show()
```

Explanation:

- Histogram shows return distribution.
 - KDE curve shows probability density.
 - Used to analyze volatility and risk.
-

Boxplot – Outlier Detection

```
plt.figure(figsize=(8,5))  
sns.boxplot(x=df['daily_return'])  
plt.title("Outliers in Daily Returns")  
plt.show()
```

Explanation:

- Identifies extreme price movements.
 - Important for financial risk analysis.
-

Correlation Heatmap

```
corr = df[['open', 'high', 'low', 'close', 'volume']].corr()  
plt.figure(figsize=(10,6))  
sns.heatmap(corr, annot=True, cmap='coolwarm')  
plt.title("Correlation Heatmap")  
plt.show()
```

Explanation:

- Shows relationship between numerical variables.
 - Strong correlation seen between open, high, low, and close prices.
 - Helps in portfolio diversification.
-

Data Storytelling**3-Chart Business Narrative****Trend Analysis**

Shows long-term stock movement.

Distribution Analysis

Explains market volatility and risk.

Relationship Analysis

Identifies variable dependencies.

FINAL WEEK 6 OUTPUT

- ✓ Line Chart (Trend)
 - ✓ Bar Chart (Comparison)
 - ✓ Scatter Plot (Relationship)
 - ✓ Pie Chart (Market Sentiment)
 - ✓ Histogram (Distribution)
 - ✓ Heatmap (Correlation)
 - ✓ Business Interpretations
-

CONCLUSION

In Week 6, I performed complete visual data analysis on financial stock market data using Python using Matplotlib and Seaborn libraries. This analysis enables investors to clearly understand overall market trends, evaluate risk exposure, observe price movement behavior, and identify relationships between different financial assets, supporting better investment and portfolio decision-making.